



Java OOP Curriculum

20~30대 초보자 대상 Java 객체지향 수업용 커리큘럼입니다.

구성 순서: 예제 설명 → 예제 코드 → 😊 토막 퀴즈!

1. 객체지향 들어가기: 클래스와 객체

1) 객체지향이란?

(1) 예제 설명

객체지향은 프로그램을 “기능 묶음”이 아니라 “현실의 대상”처럼 바라보는 방식입니다.

예를 들어 학생 관리 프로그램을 만든다고 하면, 학생 한 명은 이름, 나이, 점수 같은 정보를 가지고 있습니다. 그리고 자기소개하기, 점수 확인하기 같은 동작도 할 수 있습니다.

Java에서는 이런 대상을 만들기 위해 **클래스**와 **객체**를 사용합니다.

- 클래스: 설계도
- 객체: 설계도로 실제 만들어진 것

(2) 예제 코드

```
public class Student {  
    // 학생 객체가 기억해야 할 정보입니다.  
    // name: 학생 이름  
    // age: 학생 나이  
    // score: 학생 점수  
    String name;  
    int age;  
    int score;  
  
    // 학생 객체가 자기소개하는 기능입니다.  
    void introduce() {  
        System.out.println("안녕하세요. 저는 " + name + "입니다.");  
        System.out.println("나이는 " + age + "살입니다.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Student 클래스를 기반으로 실제 학생 객체를 하나 만듭니다.  
        Student s1 = new Student();  
    }  
}
```

```

// 객체 안에 있는 변수에 값을 넣습니다.
s1.name = "홍길동";
s1.age = 20;
s1.score = 90;

// 객체가 가진 기능을 실행합니다.
s1.introduce();
}
}

```

(3) 🤔 토막 퀴즈!

`Student` 클래스에 `study()` 메서드를 추가하고, 실행하면 아래 문장이 출력되게 해보세요.

홍길동이 공부합니다.

예시 답안

```

public class Student {
    String name;
    int age;
    int score;

    void introduce() {
        System.out.println("안녕하세요. 저는 " + name + "입니다.");
        System.out.println("나이는 " + age + "살입니다.");
    }

    void study() {
        System.out.println(name + "이 공부합니다.");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();

        s1.name = "홍길동";
        s1.age = 20;
        s1.score = 90;

        s1.introduce();
        s1.study();
    }
}

```

```
}  
}
```

2) 클래스 만들기 연습

(1) 예제 설명

이번에는 자동차를 클래스로 만들어봅시다.

자동차는 다음과 같은 정보를 가질 수 있습니다.

- 브랜드
- 색상
- 속도

그리고 다음과 같은 기능을 가질 수 있습니다.

- 달리기
- 멈추기
- 현재 속도 출력하기

(2) 예제 코드

```
public class Car {  
    // 자동차 객체가 가진 상태값입니다.  
    // brand: 자동차 브랜드  
    // color: 자동차 색상  
    // speed: 현재 속도  
    String brand;  
    String color;  
    int speed;  
  
    void drive() {  
        // drive() 메서드가 실행되면 자동차 속도를 60으로 변경합니다.  
        speed = 60;  
        System.out.println(brand + " 자동차가 달립니다.");  
    }  
  
    void stop() {  
        // stop() 메서드가 실행되면 자동차 속도를 0으로 변경합니다.  
        speed = 0;  
        System.out.println(brand + " 자동차가 멈췄습니다.");  
    }  
  
    void showSpeed() {  
        // 현재 자동차 객체가 가지고 있는 speed 값을 출력합니다.  
        System.out.println("현재 속도: " + speed + "km/h");  
    }  
}
```

```
}  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Car 클래스를 기반으로 자동차 객체를 하나 만듭니다.  
        Car car = new Car();  
  
        // 자동차 객체의 상태값을 설정합니다.  
        car.brand = "현대";  
        car.color = "검정";  
        car.speed = 0;  
  
        // 자동차를 달리게 한 뒤 현재 속도를 확인합니다.  
        car.drive();  
        car.showSpeed();  
  
        // 자동차를 멈춘 뒤 현재 속도를 다시 확인합니다.  
        car.stop();  
        car.showSpeed();  
    }  
}
```

객체지향에서 중요한 점은 객체가 자기 데이터를 가지고 있고, 그 데이터를 바꾸는 기능도 함께 가진다는 것입니다.

(3) 😊 토막 퀴즈!

① 문제

`Car` 클래스에 `speedUp()` 메서드를 추가하세요. 실행할 때마다 속도가 10씩 증가해야 합니다.

① 예시 답안

```
void speedUp() {  
    speed += 10;  
    System.out.println("속도가 10 증가했습니다.");  
}
```

② 문제

`speedDown()` 메서드를 추가하세요. 실행할 때마다 속도가 10씩 감소해야 합니다. 단, 속도는 0보다 작아지면 안 됩니다.

① 예시 답안

```
void speedDown() {
    speed -= 10;

    if (speed < 0) {
        speed = 0;
    }

    System.out.println("속도가 10 감소했습니다.");
}
```

2. 필드, 메서드, 생성자

1) 필드와 메서드 다시 정리

(1) 예제 설명

클래스 안에 있는 변수는 필드라고 부릅니다.

클래스 안에 있는 함수는 메서드라고 부릅니다.

학생 객체를 조금 더 실무에 가까운 형태로 만들어보겠습니다.

학생은 이름과 점수를 가지고 있고, 점수가 60점 이상이면 합격, 아니면 불합격으로 판단합니다.

(2) 예제 코드

```
public class Student {
    // 객체가 기억해야 할 데이터입니다.
    String name;
    int score;

    // 학생의 기본 정보를 출력하는 메서드입니다.
    void printInfo() {
        System.out.println("이름: " + name);
        System.out.println("점수: " + score);
    }

    // score 값을 기준으로 합격 여부를 판단합니다.
    void checkPass() {
        if (score >= 60) {
            System.out.println("합격입니다.");
        } else {
            System.out.println("불합격입니다.");
        }
    }
}
```

```

    }
}
}

```

```

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();

        // 학생 객체에 이름과 점수를 저장합니다.
        s1.name = "김자바";
        s1.score = 85;

        // 학생 정보 출력 후 합격 여부를 확인합니다.
        s1.printInfo();
        s1.checkPass();
    }
}

```

객체는 단순히 데이터를 담은 상자가 아니라, 그 데이터로 판단하거나 행동할 수 있습니다.

(3) 😊 토막 퀴즈!

① 문제

점수가 90점 이상이면 A, 80점 이상이면 B, 70점 이상이면 C, 그 외에는 D를 출력하는 printGrade() 메서드를 만들어보세요.

예시 답안

```

void printGrade() {
    if (score >= 90) {
        System.out.println("A");
    } else if (score >= 80) {
        System.out.println("B");
    } else if (score >= 70) {
        System.out.println("C");
    } else {
        System.out.println("D");
    }
}

```

2) 생성자 사용하기

(1) 예제 설명

지금까지는 객체를 만든 뒤에 값을 따로 넣었습니다.

```
Student s1 = new Student();  
s1.name = "김자바";  
s1.score = 85;
```

하지만 객체를 만들 때부터 필요한 값을 넣고 싶을 수 있습니다.

이때 사용하는 것이 생성자입니다.

(2) 예제 코드

```
public class Student {  
    String name;  
    int score;  
  
    // 생성자는 클래스 이름과 똑같은 이름을 가집니다.  
    // 객체가 만들어질 때 자동으로 실행됩니다.  
    Student(String name, int score) {  
        // this.name은 객체의 필드입니다.  
        // 오른쪽 name은 생성자로 전달받은 값입니다.  
        this.name = name;  
        this.score = score;  
    }  
  
    void printInfo() {  
        System.out.println("이름: " + name);  
        System.out.println("점수: " + score);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // 객체를 만들 때 이름과 점수를 바로 전달합니다.  
        Student s1 = new Student("김자바", 85);  
        Student s2 = new Student("이자바", 72);  
  
        s1.printInfo();  
        s2.printInfo();  
    }  
}
```

(3) 😊 토막 퀴즈!

① 문제

`Book` 클래스를 만들고, 제목과 가격을 생성자로 받도록 만들어보세요.

① 예시 답안

```
public class Book {  
    String title;  
    int price;  
  
    Book(String title, int price) {  
        this.title = title;  
        this.price = price;  
    }  
  
    void printInfo() {  
        System.out.println("책 제목: " + title);  
        System.out.println("가격: " + price + "원");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Book book = new Book("자바 입문", 15000);  
        book.printInfo();  
    }  
}
```

3. 캡슐화와 접근제어자

1) public, private 이해하기

(1) 예제 설명

객체의 필드를 아무 곳에서나 직접 바꾸면 문제가 생길 수 있습니다.

예를 들어 나이는 음수가 될 수 없는데, 외부에서 직접 값을 넣으면 잘못된 값이 들어갈 수 있습니다.

```
person.age = -100;
```

이런 문제를 막기 위해 필드를 `private` 으로 숨기고, 메서드를 통해서만 값을 바꾸게 합니다.

(2) 예제 코드

```
public class Person {  
    // private 필드는 클래스 밖에서 직접 접근할 수 없습니다.  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
  
        // 생성자에서도 setAge()를 사용하면 잘못된 나이를 검사할 수 있습니다.  
        setAge(age);  
    }  
  
    public void setAge(int age) {  
        // 나이가 음수로 들어오는 것을 막습니다.  
        if (age < 0) {  
            System.out.println("나이는 음수가 될 수 없습니다.");  
            return;  
        }  
  
        this.age = age;  
    }  
  
    public void printInfo() {  
        System.out.println("이름: " + name);  
        System.out.println("나이: " + age);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Person p1 = new Person("홍길동", 20);  
        p1.printInfo();  
  
        // setAge()를 통해 값을 넣으면 잘못된 값인지 검사할 수 있습니다.  
        p1.setAge(-10);  
        p1.printInfo();  
    }  
}
```

캡슐화는 필드를 숨기고, 메서드를 통해 안전하게 접근하게 만드는 방식입니다.



(3) 😊 토막 퀴즈!

① 문제

Account 클래스를 만들고, 잔액 `balance` 를 `private` 으로 선언하세요. 입금은 가능하지만 음수 입금은 막아 보세요.

① 예시 답안

```
public class Account {  
    private int balance;  
  
    public void deposit(int money) {  
        if (money <= 0) {  
            System.out.println("입금 금액은 0보다 커야 합니다.");  
            return;  
        }  
  
        balance += money;  
        System.out.println(money + "원이 입금되었습니다.");  
    }  
  
    public void printBalance() {  
        System.out.println("현재 잔액: " + balance + "원");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Account account = new Account();  
        account.deposit(10000);  
        account.deposit(-5000);  
        account.printBalance();  
    }  
}
```

4. 상속

진행 방식: 기본 틀 제공 + 핵심 키워드 직접 입력

Animal 같은 부모 클래스는 기본 틀로 제공하고, 학생은 `extends`, `super`, 자식 클래스의 고유 메서드를 직접 입력합니다.

1) 상속 기본 개념

(1) 예제 설명

상속은 기존 클래스의 필드와 메서드를 물려받아 새로운 클래스를 만드는 문법입니다.

예를 들어 `Animal` 클래스에 이름과 먹기 기능이 있다고 해봅시다.

강아지와 고양이는 모두 동물이므로, 공통 기능은 `Animal` 에 두고 각각 다른 기능만 따로 만들 수 있습니다.

(2) 예제 코드

```
public class Animal {  
    // 강아지, 고양이, 새가 공통으로 가질 이름입니다.  
    String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
  
    // 모든 동물이 공통으로 사용할 수 있는 기능입니다.  
    void eat() {  
        System.out.println(name + "이 먹습니다.");  
    }  
}
```

```
// Dog 클래스는 Animal 클래스를 상속받습니다.  
public class Dog extends Animal {  
    Dog(String name) {  
        // 부모 클래스인 Animal의 생성자를 호출합니다.  
        super(name);  
    }  
  
    // 강아지만 가진 기능입니다.  
    void bark() {  
        System.out.println(name + "이 멍멍 짩습니다.");  
    }  
}
```

```
// Cat 클래스도 Animal 클래스를 상속받습니다.  
public class Cat extends Animal {  
    Cat(String name) {  
        super(name);  
    }  
}
```

```
// 고양이만 가진 기능입니다.
void meow() {
    System.out.println(name + "이 야옹 읍니다.");
}
}
```

```
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog("초코");
        Cat cat = new Cat("나비");

        // Dog와 Cat은 eat()을 직접 만들지 않았지만 부모에게 물려받아 사용할 수 있습니다.
        dog.eat();
        dog.bark();

        cat.eat();
        cat.meow();
    }
}
```

(3) 😊 토막 퀴즈!

① 문제

`Animal` 을 상속받는 `Bird` 클래스를 만들고, `fly()` 메서드를 추가해보세요.

① 예시 답안 — 기본 틀 제공 + 핵심 키워드 직접 입력

```
public class Bird extends Animal {
    Bird(String name) {
        super(name);
    }

    void fly() {
        System.out.println(name + "이 날아갑니다.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Bird bird = new Bird("참새");

        bird.eat();
        bird.fly();
    }
}
```

```
}  
}
```

5. 오버라이딩과 다형성

진행 방식: 기본 틀 제공 + 핵심 키워드 직접 입력

부모 클래스와 일부 자식 클래스는 제공하고, 학생은 `@Override`, 오버라이딩 메서드, 다형성 변수 선언을 직접 입력합니다.

1) 오버라이딩

(1) 예제 설명

오버라이딩은 부모 클래스의 메서드를 자식 클래스에서 다시 정의하는 것입니다.

동물은 모두 소리를 낼 수 있지만, 강아지와 고양이의 소리는 다릅니다.

이럴 때 부모에는 공통 메서드를 만들고, 자식 클래스에서 각자 다르게 구현합니다.

(2) 예제 코드

```
public class Animal {  
    String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
  
    // 자식 클래스에서 다시 정의할 공통 메서드입니다.  
    void sound() {  
        System.out.println("동물이 소리를 냅니다.");  
    }  
}
```

```
public class Dog extends Animal {  
    Dog(String name) {  
        super(name);  
    }  
  
    // @Override는 부모의 메서드를 자식 클래스에서 다시 정의했다는 표시입니다.  
    @Override  
    void sound() {  
        System.out.println(name + "이 멍멍 짩습니다.");  
    }  
}
```

```
}  
}
```

```
public class Cat extends Animal {  
    Cat(String name) {  
        super(name);  
    }  
  
    @Override  
    void sound() {  
        System.out.println(name + "이 야옹 읍니다.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Dog dog = new Dog("초코");  
        Cat cat = new Cat("나비");  
  
        // 같은 sound() 메서드지만 객체에 따라 다른 결과가 나옵니다.  
        dog.sound();  
        cat.sound();  
    }  
}
```

(3) 😊 토막 퀴즈!

① 문제

`Animal` 을 상속받는 `Cow` 클래스를 만들고, `sound()` 를 오버라이딩해서 아래처럼 출력되게 해보세요.

소가 음메 읍니다.

① 예시 답안 — 기본 틀 제공 + 핵심 키워드 직접 입력

```
public class Cow extends Animal {  
    Cow(String name) {  
        super(name);  
    }  
  
    @Override  
    void sound() {  
        System.out.println(name + "가 음메 읍니다.");  
    }  
}
```

```
}  
}
```

2) 다형성

(1) 예제 설명

다형성은 “하나의 타입으로 여러 객체를 다룰 수 있는 성질”입니다.

강아지도 동물이고, 고양이도 동물입니다.

그래서 `Dog`, `Cat` 객체를 `Animal` 타입 변수에 담을 수 있습니다.

(2) 예제 코드

```
public class Main {  
    public static void main(String[] args) {  
        // 변수 타입은 Animal이지만 실제 객체는 Dog입니다.  
        Animal a1 = new Dog("초코");  
  
        // 변수 타입은 Animal이지만 실제 객체는 Cat입니다.  
        Animal a2 = new Cat("나비");  
  
        // 실행되는 메서드는 실제 객체의 sound()입니다.  
        a1.sound();  
        a2.sound();  
    }  
}
```

이것이 다형성의 핵심입니다. 하나의 부모 타입으로 여러 자식 객체를 다룰 수 있습니다.

(3) 😊 토막 퀴즈!

① 문제

`Animal[]` 배열을 만들고, `Dog`, `Cat`, `Cow` 객체를 넣은 뒤 반복문으로 `sound()`를 실행해보세요.

① 예시 답안 — 기본 틀 제공 + 핵심 키워드 직접 입력

```
public class Main {  
    public static void main(String[] args) {  
        Animal[] animals = {  
            new Dog("초코"),  
            new Cat("나비"),  
            new Cow("소")  
        };  
    }  
}
```

```

};

for (Animal animal : animals) {
    animal.sound();
}
}
}

```

6. 추상 클래스와 인터페이스

진행 방식: 기본 틀 제공 + 핵심 선언부 직접 입력

처음 예제는 `Main.java` 하나에 작성합니다. 단, Java는 한 파일에 `public class`를 하나만 둘 수 있으므로 `Main`만 `public`으로 작성하고, `Payment`, `CardPayment`, `Notifier`, `EmailNotifier`는 `public` 없이 작성합니다.

`Payment`, `Notifier` 같은 공통 틀은 먼저 제공합니다. 학생은 `CardPayment`, `EmailNotifier` 구현 클래스에서 `extends`, `implements`, `@Override`, 구현 메서드를 직접 입력합니다.

토막퀴즈에서는 `BankTransferPayment`, `KakaoNotifier`를 별도 파일로 분리해서 작성합니다.

1) 추상 클래스

(1) 예제 설명

추상 클래스는 “객체를 직접 만들기 위한 클래스”라기보다, 자식 클래스들이 공통으로 따라야 할 틀을 제공하는 클래스입니다.

예를 들어 모든 결제 수단은 결제 기능이 있어야 합니다.

하지만 카드 결제, 계좌이체, 간편결제는 실제 결제 방식이 다릅니다.

(2) 예제 코드

```

public class Main {
    public static void main(String[] args) {
        // Payment payment = new Payment(30000); 는 불가능합니다.
        // 추상 클래스는 직접 객체를 만들 수 없기 때문입니다.
        Payment payment = new CardPayment(30000);

        payment.printAmount();
        payment.pay();
    }
}

// abstract가 붙은 클래스는 직접 객체를 만들 수 없습니다.

```


// 처음에는 공통 틀로 먼저 제공합니다.

```
abstract class Payment {  
    protected int amount;
```

추상 클래스

```
    Payment(int amount) {  
        this.amount = amount;  
    }
```

아기 아기

// 추상 메서드는 자식 클래스가 반드시 구현해야 합니다.

```
    abstract void pay();
```

// 일반 메서드는 추상 클래스 안에서도 만들 수 있습니다.

```
    void printAmount() {  
        System.out.println("결제 금액: " + amount + "원");  
    }  
}
```

// 학생이 직접 작성하는 구현 클래스입니다.

// extends, super, @Override, pay() 구현을 직접 입력합니다.

```
class CardPayment extends Payment {
```

```
    CardPayment(int amount) {
```

```
        super(amount);
```

아기 아기

```
    @Override
```

```
    void pay() {
```

```
        System.out.println("카드로 " + amount + "원을 결제합니다.");
```

```
    }
```

```
}
```

(3) 😊 토막 퀴즈!

① 문제

Payment 를 상속받는 BankTransferPayment 클래스를 만들고, 계좌이체 결제 문구를 출력해보세요.

① 예시 답안 — 기본 틀 제공 + 핵심 키워드 직접 입력

```
public class BankTransferPayment extends Payment {
```

```
    BankTransferPayment(int amount) {
```

```
        super(amount);
```

```
    }
```

```
    @Override
```

```
    void pay() {
```

```
        System.out.println("계좌이체로 " + amount + "원을 결제합니다.");
```

```
}  
}
```

2) 인터페이스

(1) 예제 설명

인터페이스는 클래스가 반드시 구현해야 하는 기능 목록을 정해주는 문법입니다.

예를 들어 알림 기능을 만들 때, 이메일 알림과 문자 알림은 방식은 다르지만 둘 다 send() 기능은 필요합니다.

(2) 예제 코드

```
public class Main {  
    public static void main(String[] args) {  
        // Notifier 타입으로 EmailNotifier 객체를 사용할 수 있습니다.  
        Notifier notifier = new EmailNotifier();  
        notifier.send("회원가입을 축하합니다.");  
  
        // 같은 변수에 SmsNotifier 객체도 담을 수 있습니다.  
        notifier = new SmsNotifier();  
        notifier.send("인증번호는 1234입니다.");  
    }  
}  
  
// 인터페이스는 구현해야 할 기능의 이름을 정해줍니다.  
// 처음에는 공통 틀만 먼저 제공합니다.  
interface Notifier {  
    void send(String message);  
}  
  
// 학생이 직접 작성하는 구현 클래스입니다.  
// implements, @Override, public void send()를 직접 입력합니다.  
class EmailNotifier implements Notifier {  
    @Override  
    public void send(String message) {  
        System.out.println("이메일 발송: " + message);  
    }  
}  
  
class SmsNotifier implements Notifier {  
    @Override  
    public void send(String message) {  
        System.out.println("문자 발송: " + message);  
    }  
}
```

```
}  
}
```

인터페이스를 사용하면 알림 방식이 바뀌어도 사용하는 쪽 코드는 크게 바뀌지 않습니다.

(3) 😊 토막 퀴즈!

① 문제

`KakaoNotifier` 클래스를 만들고 `Notifier` 를 구현해보세요.

① 예시 답안 — 기본 틀 제공 + 핵심 키워드 직접 입력

```
public class KakaoNotifier implements Notifier {  
    @Override  
    public void send(String message) {  
        System.out.println("카카오톡 발송: " + message);  
    }  
}
```

3) 추천 시간 배분

6번은 초보자에게 난이도가 높은 편이므로, 설명만 빠르게 지나가기보다 실습과 오류 수정까지 포함해서 진행합니다.

3시간 기준

0:00~0:20

상속 복습

- Animal, Dog 예제 다시 보기
- “공통 기능을 부모에 둔다”는 개념 복습

0:20~1:20

추상 클래스

- Payment 예제 설명
- Main.java 하나에 코드 작성
- abstract class, abstract method 설명
- CardPayment 직접 작성
- BankTransferPayment 토막퀴즈

1:20~1:30

쉬는 시간

1:30~2:30

인터페이스

- Notifier 예제 설명

- interface, implements 설명
- EmailNotifier 직접 작성
- SmsNotifier 추가
- KakaoNotifier 토막퀴즈

2:30~3:00

비교 정리

- 추상 클래스 vs 인터페이스
- extends vs implements
- 언제 뭘 쓰는지
- 오류 일부러 내보기

6시간 수업으로 운영할 경우

전반 3시간

- 추상 클래스와 인터페이스 기본 예제 진행
- Payment, Notifier 예제 작성
- BankTransferPayment, KakaoNotifier 토막퀴즈

후반 2시간

- 4~6번 복습 문제
- Animal 추상화하기
- Payment 구현체 추가하기
- Notifier 구현체 추가하기

마지막 1시간

- 오류 수정 실습
- abstract 빼먹기
- implements 했는데 메서드 구현 안 하기
- public 빠진 send()
- Main 말고 다른 class에 public 붙여보기

4) 자주 막히는 포인트 정리

① abstract class는 왜 new 할 수 없나요?

추상 클래스는 완성된 객체를 만들기 위한 클래스가 아니라, 자식 클래스들이 따라야 할 공통 틀입니다.

예를 들어 `Payment` 는 “결제한다”는 규칙은 가지고 있지만, 실제로 카드 결제인지 계좌이체인지 알 수 없습니다.

```
Payment payment = new Payment(30000); // 불가능
```

`pay()` 가 아직 구체적으로 구현되지 않았기 때문에 직접 객체를 만들 수 없습니다.

대신 자식 클래스에서 `pay()` 를 완성한 뒤 객체를 만듭니다.

```
Payment payment = new CardPayment(30000); // 가능
```

② `abstract void pay();`는 왜 중괄호가 없나요?

추상 메서드는 “이런 기능이 반드시 있어야 한다”는 약속만 정합니다.

그래서 메서드 이름, 매개변수, 반환 타입만 있고 실제 실행 코드는 없습니다.

```
abstract void pay();
```

실제 동작은 자식 클래스가 정합니다.

```
@Override
void pay() {
    System.out.println("카드로 결제합니다.");
}
```

③ `extends`와 `implements`는 뭐가 다른가요?

`extends` 는 클래스를 상속받을 때 사용합니다.

```
class CardPayment extends Payment {
}
```

`implements` 는 인터페이스를 구현할 때 사용합니다.

```
class EmailNotifier implements Notifier {
}
```

간단히 정리하면 다음과 같습니다.

```
extends    → 부모 클래스를 물려받는다
implements → 인터페이스의 약속을 지킨다
```

④ 인터페이스 메서드는 왜 public으로 구현해야 하나요?

인터페이스의 메서드는 기본적으로 `public` 성격을 가집니다.

그래서 구현 클래스에서 접근 범위를 더 좁게 만들 수 없습니다.

잘못된 예:

```
class EmailNotifier implements Notifier {  
    @Override  
    void send(String message) { // 오류 가능  
        System.out.println(message);  
    }  
}
```

올바른 예:

```
class EmailNotifier implements Notifier {  
    @Override  
    public void send(String message) {  
        System.out.println(message);  
    }  
}
```

⑤ 한 파일에 class를 여러 개 넣어도 되나요?

가능합니다.

다만 Java에서는 한 파일 안에 `public class` 를 하나만 둘 수 있습니다.

그래서 `Main.java` 하나에 여러 클래스를 넣을 때는 Main 만 public 으로 작성합니다.

```
public class Main {  
}  
  
class Dog {  
}  
  
class Cat {  
}
```

잘못된 예:

```
public class Main {
}

public class Dog { // 오류 가능
}
```

5) 복습 문제: Animal 추상화하기

난이도: **중상**

이 문제는 4~6번 내용을 모두 합친 종합 복습 문제입니다. 처음부터 안 보고 전체 코드를 작성하기는 어렵습니다.

진행은 1단계 빈칸 채우기 → 2단계 일부 클래스 작성 → 3단계 전체 코드 다시 작성 순서로 합니다.

(1) 문제 설명

기존 `Animal` 예제는 부모 클래스의 `sound()` 메서드에 기본 출력문이 있었습니다.

```
void sound() {
    System.out.println("동물이 소리를 냅니다.");
}
```

하지만 실제로는 동물마다 소리가 다르기 때문에, 부모 클래스에서 소리를 정해두는 것보다 자식 클래스가 반드시 직접 구현하게 만드는 것이 더 자연스럽습니다.

`Animal` 을 추상 클래스로 바꾸고, `sound()` 를 추상 메서드로 바꿔보세요.

(2) 요구사항

- `Animal` 은 추상 클래스여야 합니다.
- `sound()` 는 추상 메서드여야 합니다.
- `Dog` , `Cat` , `Cow` 는 `Animal` 을 상속받아야 합니다.
- 각 클래스는 `sound()` 를 반드시 오버라이딩해야 합니다.
- `Main` 에서 `Animal[]` 배열을 사용해 반복문으로 `sound()` 를 실행합니다.

(3) 단계별 진행 방식

① 1단계: 빈칸 채우기

먼저 핵심 키워드만 채우게 합니다.

```
abstract class Animal {
    protected String name;

    Animal(String name) {
        this.name = name;
    }
}
```

```

    }

    abstract void _____();
}

```

정답:

sound

```

class Dog _____ Animal {
    Dog(String name) {
        _____(name);
    }

    _____

    void sound() {
        System.out.println(name + "이 멍멍 짩습니다.");
    }
}

```

정답:

extends
super
@Override

② 2단계: Cow 클래스만 직접 작성하기

`Animal`, `Dog`, `Cat`, `Main` 은 제공하고, 학생은 `Cow` 클래스만 직접 작성합니다.

```

class Cow extends Animal {
    Cow(String name) {
        super(name);
    }

    @Override
    void sound() {
        System.out.println(name + "가 음메 읊니다.");
    }
}

```

③ 3단계: 전체 코드 다시 작성하기

1단계와 2단계를 마친 뒤, 아래 답안을 보지 않고 다시 작성해봅니다.

완전히 안 보고 작성하는 것이 어렵다면 다음 순서로 힌트를 제공합니다.

1. Main의 Animal[] 배열만 먼저 보여주기
2. abstract class Animal 틀만 보여주기
3. Dog 클래스 하나만 보여주기
4. Cat, Cow는 Dog를 참고해서 작성하기

(4) 예시 답안 코드

```
public class Main {
    public static void main(String[] args) {
        Animal[] animals = {
            new Dog("초코"),
            new Cat("나비"),
            new Cow("소")
        };

        for (Animal animal : animals) {
            animal.sound();
        }
    }
}

// Animal은 직접 객체를 만들 수 없는 추상 클래스입니다.
abstract class Animal {
    protected String name;

    Animal(String name) {
        this.name = name;
    }

    // 동물마다 소리가 다르므로 자식 클래스가 반드시 구현하게 만듭니다.
    abstract void sound();
}

class Dog extends Animal {
    Dog(String name) {
        super(name);
    }

    @Override
    void sound() {
        System.out.println(name + "이 멍멍 짭니다.");
    }
}
```

```

class Cat extends Animal {
    Cat(String name) {
        super(name);
    }

    @Override
    void sound() {
        System.out.println(name + "이 야옹 읍니다.");
    }
}

class Cow extends Animal {
    Cow(String name) {
        super(name);
    }

    @Override
    void sound() {
        System.out.println(name + "가 음메 읍니다.");
    }
}

```

(5) 추가 오류 수정 실습

아래 오류를 일부러 만들어보고, 왜 오류인지 확인합니다.

1. abstract class Animal에서 abstract를 지우기
2. Dog 클래스에서 sound()를 구현하지 않기
3. @Override를 붙였는데 메서드 이름을 sounds()로 잘못 쓰기
4. Main 말고 Dog에도 public을 붙이기

7. 실무형 객체 설계 연습

진행 방식: 자동 틀 제공 + 기능 추가/오류 수정

Member, MemberManager, Main의 기본 파일 구조는 자동으로 제공하고, 학생은 빠진 메서드나 조건식을 채웁니다. 이후 일부러 잘못된 조건식, 누락된 getter, 잘못된 반복 범위를 넣어 오류를 찾고 수정하게 합니다.

VS Code 자동 생성 기능 사용 방법

1. Member.java, MemberManager.java, Main.java 파일을 먼저 만듭니다.
2. Member.java에는 필드까지만 먼저 직접 입력합니다.

```
public class Member {
    private String id;
    private String name;
    private int age;
}
```

3. 클래스 안쪽 빈 공간에서 **우측 마우스 클릭 → Source Action... → Generate Constructors... **를 클릭해 생성자를 자동 생성합니다.
4. 다시 클래스 안쪽 빈 공간에서 **우측 마우스 클릭 → Source Action... → Generate Getters and Setters. **를 클릭합니다.
5. id, name, age 중 필요한 필드를 체크하고 getter/setter를 자동 생성합니다.
6. printInfo() 같은 직접 구현 메서드는 학생이 손으로 작성합니다.
7. MemberManager.java, Main.java 는 클래스와 필드까지만 먼저 입력하고, 메서드 내부 로직은 수업 중 직접 채웁니다.

1) 회원 관리 프로그램 만들기

(1) 예제 설명

이번에는 객체지향 문법을 모아서 간단한 회원 관리 프로그램을 만들어봅시다.

회원은 다음 정보를 가집니다.

- 아이디
- 이름
- 나이

회원 관리자는 다음 기능을 가집니다.

- 회원 등록
- 회원 목록 출력

(2) 예제 코드

```
// Member는 회원 한 명의 정보를 표현하는 클래스입니다.
public class Member {
    private String id;
    private String name;
    private int age;

    public Member(String id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    public void printInfo() {
        System.out.println("아이디: " + id + ", 이름: " + name + ", 나이: " + age);
    }
}
```

```
}  
}
```

```
// MemberManager는 여러 회원을 관리하는 클래스입니다.  
public class MemberManager {  
    // 회원을 최대 10명까지 저장하는 배열입니다.  
    private Member[] members = new Member[10];  
  
    // 현재 등록된 회원 수입니다.  
    private int count = 0;  
  
    public void addMember(Member member) {  
        // 배열이 가득 차면 더 이상 등록하지 않습니다.  
        if (count >= members.length) {  
            System.out.println("더 이상 회원을 등록할 수 없습니다.");  
            return;  
        }  
  
        members[count] = member;  
        count++;  
        System.out.println("회원이 등록되었습니다.");  
    }  
  
    public void printMembers() {  
        System.out.println("회원 목록");  
  
        // 등록된 회원 수만큼만 반복합니다.  
        for (int i = 0; i < count; i++) {  
            members[i].printInfo();  
        }  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // 프로그램 실행 담당 클래스입니다.  
        MemberManager manager = new MemberManager();  
  
        // 회원 객체를 만들어 관리자에게 등록합니다.  
        manager.addMember(new Member("user1", "홍길동", 20));  
        manager.addMember(new Member("user2", "김자바", 25));  
  
        manager.printMembers();  
    }  
}
```

객체지향 설계에서는 한 클래스가 모든 일을 하지 않게 나누는 것이 중요합니다.

(3) 😊 토막 퀴즈!

① 문제

`MemberManager` 에 `findMemberById(String id)` 메서드를 추가해보세요.

단, `Member` 의 `id` 는 `private`이므로 `getId()` 메서드가 필요합니다.

① 예시 답안 — 자동 틀 제공 + 기능 추가/오류 수정

```
public class Member {  
    private String id;  
    private String name;  
    private int age;  
  
    public Member(String id, String name, int age) {  
        this.id = id;  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getId() {  
        return id;  
    }  
  
    public void printInfo() {  
        System.out.println("아이디: " + id + ", 이름: " + name + ", 나이: " + age);  
    }  
}
```

```
public Member findMemberById(String id) {  
    for (int i = 0; i < count; i++) {  
        if (members[i].getId().equals(id)) {  
            return members[i];  
        }  
    }  
  
    return null;  
}
```

8. ArrayList로 객체 관리하기

진행 방식: 자동 틀 제공 + 기능 추가/오류 수정

배열 버전 코드를 먼저 제공한 뒤, ArrayList 버전의 틀을 자동으로 넣어줍니다. 학생은 add, size, get, remove, 향상된 for문을 직접 채우고, 인덱스 오류나 삭제 로직 오류를 수정합니다.

VS Code 자동 생성 기능 사용 방법

1. 7번에서 만든 배열 버전 MemberManager 코드를 먼저 열어둡니다.
2. private Member[] members = new Member[10]; 부분을 학생이 직접 ArrayList로 바꾸게 합니다.

```
private ArrayList<Member> members = new ArrayList<>();
```

3. ArrayList에 빨간 줄이 뜨면 ArrayList 위에 커서를 올리고 **Quick Fix... → Import 'ArrayList' (java.util)**를 클릭합니다.
4. 또는 파일 상단 빈 줄에서 직접 import java.util.ArrayList; 를 입력합니다.
5. 배열에서 쓰던 members.length, members[i] 는 자동 생성 메뉴가 아니라 학생이 직접 members.size(), members.get(i) 로 고칩니다.
6. 반복문 작성이 막히면 빈 줄에서 for 를 입력한 뒤 VS Code 자동완성 목록에서 for 또는 foreach 템플릿을 선택해 기본 반복문 틀을 생성합니다.
7. 삭제 기능은 메서드 이름과 매개변수까지만 먼저 입력하고, 내부의 remove() 로직은 학생이 직접 작성합니다.

1) 배열에서 ArrayList로 변경하기

(1) 예제 설명

배열은 크기가 고정되어 있습니다.

```
private Member[] members = new Member[10];
```

회원이 10명을 넘으면 더 이상 저장할 수 없습니다.

실무에서는 크기가 자동으로 늘어나는 ArrayList 를 많이 사용합니다.

(2) 예제 코드

```
import java.util.ArrayList;

public class MemberManager {
    // Member 객체만 저장하는 ArrayList입니다.
    // 배열과 다르게 크기가 자동으로 늘어납니다.
    private ArrayList<Member> members = new ArrayList<>();

    public void addMember(Member member) {
```

```

// 리스트에 회원 객체를 추가합니다.
members.add(member);
System.out.println("회원이 등록되었습니다.");
}

public void printMembers() {
    System.out.println("회원 목록");

    // 향상된 for문으로 리스트 안의 회원을 하나씩 꺼냅니다.
    for (Member member : members) {
        member.printInfo();
    }
}

public Member findMemberById(String id) {
    for (Member member : members) {
        if (member.getId().equals(id)) {
            return member;
        }
    }

    return null;
}
}

```

```

public class Main {
    public static void main(String[] args) {
        MemberManager manager = new MemberManager();

        manager.addMember(new Member("user1", "홍길동", 20));
        manager.addMember(new Member("user2", "김자바", 25));
        manager.addMember(new Member("user3", "이자바", 30));

        manager.printMembers();

        // 아이디가 user2인 회원을 검색합니다.
        Member found = manager.findMemberById("user2");

        if (found != null) {
            System.out.println("검색 결과");
            found.printInfo();
        } else {
            System.out.println("회원을 찾을 수 없습니다.");
        }
    }
}

```

(3) 😊 토막 퀴즈!

① 문제

`removeMemberById(String id)` 메서드를 추가해서 아이디가 일치하는 회원을 삭제해보세요.

① 예시 답안 — 자동 틀 제공 + 기능 추가/오류 수정

```
public void removeMemberById(String id) {  
    for (int i = 0; i < members.size(); i++) {  
        if (members.get(i).getId().equals(id)) {  
            members.remove(i);  
            System.out.println("회원이 삭제되었습니다.");  
            return;  
        }  
    }  
  
    System.out.println("삭제할 회원을 찾을 수 없습니다.");  
}
```

9. 예외 처리와 객체지향 프로그램 안정화

진행 방식: 자동 틀 제공 + 기능 추가/오류 수정

예외가 발생하는 코드를 먼저 자동으로 제공하고, 학생은 `try-catch` 구조를 추가합니다. 이후 잘못된 예외 타입, 빠진 `catch`, 잘못된 입력값 처리 등을 고치며 예외 처리 감각을 익힙니다.

VS Code 자동 생성 기능 사용 방법

1. `Main.java` 에 예외가 발생할 수 있는 코드만 먼저 입력합니다. 예: `int result = 10 / 0;` 또는 `Integer.parseInt("abc");`
2. 해당 코드를 블록 지정합니다.
3. 우측 마우스 클릭 → **Surround With...** → **try/catch**를 클릭해 `try-catch` 틀을 자동 생성합니다.
4. 메뉴가 보이지 않으면 블록 지정 후 `Ctrl + Shift + P` 를 누르고 `Surround With` 를 검색해서 선택합니다.
5. 자동 생성된 `catch` 안의 출력 문장은 학생이 직접 작성합니다.
6. 예외 타입이 너무 넓게 잡히면 `Exception` 을 `ArithmeticException`, `NumberFormatException` 처럼 상황에 맞게 바꾸는 연습을 합니다.
7. 빨간 줄이 뜨는 경우 커서를 올리고 **Quick Fix...**를 눌러 VS Code가 제안하는 수정 내용을 확인합니다.

1) 예외 처리 기본

(1) 예제 설명

프로그램 실행 중 문제가 생기는 상황을 예외라고 합니다.

예를 들어 숫자를 0으로 나누거나, 존재하지 않는 회원을 사용하려고 하면 문제가 생길 수 있습니다.

예외 처리를 하면 프로그램이 갑자기 종료되지 않고, 사용자에게 안내 메시지를 보여줄 수 있습니다.

(2) 예제 코드

```
public class Main {
    public static void main(String[] args) {
        try {
            // 문제가 생길 수 있는 코드를 try 안에 넣습니다.
            int result = 10 / 0;
            System.out.println(result);
        } catch (ArithmeticException e) {
            // 문제가 발생하면 catch 안의 코드가 실행됩니다.
            System.out.println("0으로 나눌 수 없습니다.");
        }

        // 예외 처리를 했기 때문에 프로그램이 중간에 종료되지 않습니다.
        System.out.println("프로그램이 계속 실행됩니다.");
    }
}
```

(3) 😊 토막 퀴즈!

① 문제

문자열을 숫자로 바꾸는 코드에서 예외가 발생하면 `숫자로 변환할 수 없습니다.`를 출력해보세요.

① 예시 답안 — 자동 틀 제공 + 기능 추가/오류 수정

```
public class Main {
    public static void main(String[] args) {
        try {
            String text = "abc";
            int number = Integer.parseInt(text);
            System.out.println(number);
        } catch (NumberFormatException e) {
            System.out.println("숫자로 변환할 수 없습니다.");
        }
    }
}
```

10. 미니 프로젝트: 콘솔 회원 관리 프로그램

진행 방식: 자동 틀 제공 + 기능 추가/오류 수정

전체 프로젝트 파일과 메뉴 구조는 자동으로 제공하고, 학생은 검색, 삭제, 수정, 번호 출력 같은 기능을 추가합니다. 이후 일부러 누락된 getter, 잘못된 메뉴 분기, `scanner.nextLine()` 누락 같은 오류를 찾아 수정하게 합니다.

VS Code 자동 생성 기능 사용 방법

1. `Member.java`, `MemberManager.java`, `Main.java` 파일을 먼저 생성합니다.
2. `Member.java` 에는 필드를 먼저 입력합니다.

```
private String id;  
private String name;  
private int age;
```

3. 클래스 안쪽 빈 공간에서 **우측 마우스 클릭 → Source Action... → Generate Constructors...**를 클릭해 생성자를 만듭니다.
4. 다시 **우측 마우스 클릭 → Source Action... → Generate Getters and Setters...**를 클릭해 `getId()`, `getName()`, `getAge()`, 필요한 setter를 생성합니다.
5. `Main.java` 에서 `Scanner` 를 입력했을 때 빨간 줄이 뜨면 `Scanner` 위에 커서를 올리고 **Quick Fix... → Import 'Scanner' (java.util)**를 클릭합니다.
6. 메뉴 반복 구조는 `while` 을 입력한 뒤 VS Code 자동완성 목록에서 `while` 템플릿을 선택해 기본 틀을 만듭니다.
7. 조건문은 `if` 를 입력한 뒤 자동완성 템플릿을 선택하고, `menu == 1`, `menu == 2` 같은 조건은 학생이 직접 작성합니다.
8. `MemberManager` 의 검색, 삭제, 수정 기능은 메서드 이름까지만 먼저 만들고 내부 로직은 학생이 직접 채웁니다.

1) 프로젝트 목표

(1) 예제 설명

지금까지 배운 내용을 합쳐서 콘솔 회원 관리 프로그램을 만듭니다.

기능은 다음과 같습니다.

- 회원 등록
- 회원 목록 조회
- 회원 검색
- 회원 삭제
- 프로그램 종료

사용 문법은 다음과 같습니다.

- 클래스와 객체
- 생성자
- private 필드

- getter
- ArrayList
- 반복문
- 조건문
- Scanner

2) 프로젝트 답안 코드

Member.java

```
public class Member {
    // 외부에서 직접 바꾸지 못하도록 private으로 선언합니다.
    private String id;
    private String name;
    private int age;

    public Member(String id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    // 아이디 검색을 위해 id 값을 밖으로 전달하는 getter입니다.
    public String getId() {
        return id;
    }

    public void printInfo() {
        System.out.println("아이디: " + id + ", 이름: " + name + ", 나이: " + age);
    }
}
```

MemberManager.java

```
import java.util.ArrayList;

public class MemberManager {
    // 여러 회원을 저장하는 ArrayList입니다.
    private ArrayList<Member> members = new ArrayList<>();

    public void addMember(Member member) {
        // 같은 아이디가 이미 있으면 등록하지 않습니다.
        if (findMemberById(member.getId()) != null) {
            System.out.println("이미 존재하는 아이디입니다.");
            return;
        }
    }
}
```

```

    }

    members.add(member);
    System.out.println("회원이 등록되었습니다.");
}

public void printMembers() {
    // 회원이 한 명도 없을 때 안내 메시지를 출력합니다.
    if (members.isEmpty()) {
        System.out.println("등록된 회원이 없습니다.");
        return;
    }

    System.out.println("회원 목록");
    for (Member member : members) {
        member.printInfo();
    }
}

public Member findMemberById(String id) {
    // 리스트 안의 회원을 하나씩 확인합니다.
    for (Member member : members) {
        if (member.getId().equals(id)) {
            return member;
        }
    }

    return null;
}

public void removeMemberById(String id) {
    for (int i = 0; i < members.size(); i++) {
        if (members.get(i).getId().equals(id)) {
            members.remove(i);
            System.out.println("회원이 삭제되었습니다.");
            return;
        }
    }

    System.out.println("삭제할 회원을 찾을 수 없습니다.");
}
}

```

Main.java

```
import java.util.Scanner;
```

```

public class Main {
    public static void main(String[] args) {
        // 사용자 입력을 받기 위한 Scanner입니다.
        Scanner scanner = new Scanner(System.in);

        // 회원 관리 기능을 담당하는 객체입니다.
        MemberManager manager = new MemberManager();

        while (true) {
            System.out.println();
            System.out.println("===== 회원 관리 프로그램 =====");
            System.out.println("1. 회원 등록");
            System.out.println("2. 회원 목록 조회");
            System.out.println("3. 회원 검색");
            System.out.println("4. 회원 삭제");
            System.out.println("0. 종료");
            System.out.print("메뉴 선택: ");

            int menu = scanner.nextInt();
            scanner.nextLine(); // 숫자 입력 후 남아 있는 엔터를 제거합니다.

            if (menu == 1) {
                System.out.print("아이디: ");
                String id = scanner.nextLine();

                System.out.print("이름: ");
                String name = scanner.nextLine();

                System.out.print("나이: ");
                int age = scanner.nextInt();
                scanner.nextLine();

                Member member = new Member(id, name, age);
                manager.addMember(member);
            } else if (menu == 2) {
                manager.printMembers();
            } else if (menu == 3) {
                System.out.print("검색할 아이디: ");
                String id = scanner.nextLine();

                Member found = manager.findMemberById(id);

                if (found != null) {
                    found.printInfo();
                } else {
                    System.out.println("회원을 찾을 수 없습니다.");
                }
            }
        }
    }
}

```

```

    }

    } else if (menu == 4) {
        System.out.print("삭제할 아이디: ");
        String id = scanner.nextLine();
        manager.removeMemberById(id);

    } else if (menu == 0) {
        System.out.println("프로그램을 종료합니다.");
        break;

    } else {
        System.out.println("잘못된 메뉴입니다.");
    }
}

scanner.close();
}
}

```

이번 프로젝트는 `Member`, `MemberManager`, `Main` 으로 역할을 나누어 구성했습니다. 이 구조는 실무에서도 자주 사용하는 역할 분리의 가장 기본 형태입니다.

(3) 😊 토막 퀴즈!

① 문제

회원 이름으로 검색하는 기능을 추가해보세요.

② 문제

회원 나이를 수정하는 기능을 추가해보세요.

③ 문제

회원 목록을 출력할 때 번호도 함께 출력되도록 바꿔보세요.

① 예시 답안: 이름 검색 — 자동 틀 제공 + 기능 추가/오류 수정

```

public void findMembersByName(String name) {
    boolean found = false;

    for (Member member : members) {
        if (member.getName().equals(name)) {
            member.printInfo();
            found = true;
        }
    }
}

```

```

if (!found) {
    System.out.println("해당 이름의 회원을 찾을 수 없습니다.");
}
}

```

위 코드를 사용하려면 `Member` 클래스에 `getName()` 메서드가 필요합니다.

```

public String getName() {
    return name;
}

```

② 예시 답안: 번호 출력 — 자동 틀 제공 + 기능 추가/오류 수정

```

public void printMembers() {
    if (members.isEmpty()) {
        System.out.println("등록된 회원이 없습니다.");
        return;
    }

    System.out.println("회원 목록");

    for (int i = 0; i < members.size(); i++) {
        System.out.print((i + 1) + ". ");
        members.get(i).printInfo();
    }
}

```

부록. 객체지향 숙제/복습 문제 선별

교재의 모든 객체지향 문제를 풀 필요는 없습니다.

아래 문제들은 수업 내용을 복습하기 위한 쉬운 문제 위주입니다.

기준은 “문법 암기”보다 클래스를 만들고, 객체를 생성하고, 메서드를 호출할 수 있는가입니다.

1. 클래스와 객체 복습

1) 학생 클래스 만들기

문제

`Student` 클래스를 만들고 다음 정보를 저장하세요.

- 이름

- 나이
- 점수

그리고 학생 정보를 출력하는 `printInfo()` 메서드를 만드세요.

체크 포인트

- 클래스를 만들 수 있는가?
- 필드를 선언할 수 있는가?
- 객체를 생성할 수 있는가?
- 메서드를 호출할 수 있는가?

예시 답안

```
public class Student {
    String name;
    int age;
    int score;

    void printInfo() {
        System.out.println("이름: " + name);
        System.out.println("나이: " + age);
        System.out.println("점수: " + score);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Student student = new Student();

        student.name = "홍길동";
        student.age = 20;
        student.score = 90;

        student.printInfo();
    }
}
```


2. 메서드 복습

1) 계산기 클래스 만들기

문제

`Calculator` 클래스를 만들고 다음 메서드를 작성하세요.

- `add(int a, int b)` : 두 수의 합 출력
- `minus(int a, int b)` : 두 수의 차 출력

체크 포인트

- 매개변수가 있는 메서드를 만들 수 있는가?
- 메서드 안에서 계산 결과를 출력할 수 있는가?

예시 답안

```
public class Calculator {  
    void add(int a, int b) {  
        System.out.println("더하기 결과: " + (a + b));  
    }  
  
    void minus(int a, int b) {  
        System.out.println("빼기 결과: " + (a - b));  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Calculator calculator = new Calculator();  
  
        calculator.add(10, 5);  
        calculator.minus(10, 5);  
    }  
}
```

3. 생성자 복습

1) 책 클래스 만들기

문제

`Book` 클래스를 만들고, 생성자를 통해 제목과 가격을 입력받으세요.

그리고 책 정보를 출력하는 `printInfo()` 메서드를 작성하세요.

체크 포인트

- 생성자를 만들 수 있는가?
- `this` 를 사용할 수 있는가?
- 객체 생성 시 값을 전달할 수 있는가?

예시 답안

```
public class Book {
    String title;
    int price;

    Book(String title, int price) {
        this.title = title;
        this.price = price;
    }

    void printInfo() {
        System.out.println("책 제목: " + title);
        System.out.println("가격: " + price + "원");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Book book = new Book("자바 입문", 15000);
        book.printInfo();
    }
}
```

4. 캡슐화 복습

1) 계좌 클래스 만들기

문제

`Account` 클래스를 만들고 잔액 `balance` 를 `private` 으로 선언하세요.

다음 기능을 만드세요.

- 입금하기
- 현재 잔액 출력하기

단, 0원 이하의 금액은 입금할 수 없게 하세요.

체크 포인트

- `private` 필드를 사용할 수 있는가?
- 메서드로 안전하게 값을 변경할 수 있는가?
- 조건문으로 잘못된 값을 막을 수 있는가?

예시 답안

```
public class Account {
    private int balance;

    public void deposit(int money) {
        if (money <= 0) {
            System.out.println("입금 금액은 0보다 커야 합니다.");
            return;
        }

        balance += money;
        System.out.println(money + "원이 입금되었습니다.");
    }

    public void printBalance() {
        System.out.println("현재 잔액: " + balance + "원");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Account account = new Account();

        account.deposit(10000);
        account.deposit(-5000);
        account.printBalance();
    }
}
```

5. getter/setter 복습

1) 회원 클래스 만들기

문제

`Member` 클래스를 만들고 다음 필드를 `private`으로 선언하세요.

- 아이디

- 이름
- 나이

그리고 VS Code 자동 생성 기능을 사용해서 생성자와 getter/setter를 만들어보세요.

VS Code 사용 방법

1. 필드를 먼저 작성한다.
2. 클래스 안쪽 빈 공간에서 우측 마우스 클릭
3. Source Action... 클릭
4. Generate Constructors... 클릭
5. 다시 Source Action... 클릭
6. Generate Getters and Setters... 클릭

체크 포인트

- 필드를 `private` 으로 만들 수 있는가?
- 생성자를 자동 생성할 수 있는가?
- getter/setter를 자동 생성할 수 있는가?

예시 답안

```
public class Member {
    private String id;
    private String name;
    private int age;

    public Member(String id, String name, int age) {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

```

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}
}

```

6. 상속 복습

1) 동물 클래스 상속하기

문제

`Animal` 클래스를 만들고, 이를 상속받는 `Dog` 클래스를 만드세요.

`Animal` 에는 `name` 필드와 `eat()` 메서드가 있어야 합니다.

`Dog` 에는 `bark()` 메서드를 추가하세요.

체크 포인트

- `extends` 를 사용할 수 있는가?
- 부모 생성자를 `super()` 로 호출할 수 있는가?
- 부모 메서드를 자식 객체에서 사용할 수 있는가?

예시 답안

```

class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    void eat() {
        System.out.println(name + "이 먹습니다.");
    }
}

```

```

class Dog extends Animal {
    Dog(String name) {
        super(name);
    }
}

```

```

    }

    void bark() {
        System.out.println(name + "이 멍멍 짭니다.");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog("초코");

        dog.eat();
        dog.bark();
    }
}

```

7. 오버라이딩 복습

1) 동물 소리 바꾸기

문제

`Animal` 클래스에 `sound()` 메서드를 만들고, `Dog` 클래스에서 오버라이딩하세요.

체크 포인트

- 부모 메서드를 자식 클래스에서 다시 정의할 수 있는가?
- `@Override`를 사용할 수 있는가?

예시 답안

```

class Animal {
    void sound() {
        System.out.println("동물이 소리를 냅니다.");
    }
}

```

```

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("강아지가 멍멍 짭니다.");
    }
}

```

```
public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.sound();
    }
}
```

8. 다형성 복습

1) Animal 배열 사용하기

문제

`Animal[]` 배열에 `Dog`, `Cat` 객체를 넣고 반복문으로 `sound()` 를 실행하세요.

체크 포인트

- 부모 타입 배열에 자식 객체를 넣을 수 있는가?
- 반복문으로 같은 메서드를 호출할 수 있는가?

예시 답안

```
class Animal {
    void sound() {
        System.out.println("동물이 소리를 냅니다.");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("강아지가 멍멍 짭니다.");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("고양이가 야옹 읊니다.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Animal[] animals = {
```

```

        new Dog(),
        new Cat()
    };

    for (Animal animal : animals) {
        animal.sound();
    }
}
}

```

9. 추상 클래스 복습

1) 결제 클래스 만들기

문제

`Payment` 추상 클래스를 만들고, `CardPayment` 클래스에서 `pay()` 메서드를 구현하세요.

체크 포인트

- `abstract class` 를 만들 수 있는가?
- 추상 메서드를 만들 수 있는가?
- 자식 클래스에서 추상 메서드를 구현할 수 있는가?

예시 답안

```

abstract class Payment {
    abstract void pay();
}

```

```

class CardPayment extends Payment {
    @Override
    void pay() {
        System.out.println("카드 결제합니다.");
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Payment payment = new CardPayment();
        payment.pay();
    }
}

```


10. 인터페이스 복습

1) 알림 기능 만들기

문제

`Notifier` 인터페이스를 만들고, `EmailNotifier` 클래스에서 `send()` 메서드를 구현하세요.

체크 포인트

- `interface` 를 만들 수 있는가?
- `implements` 를 사용할 수 있는가?
- 인터페이스 메서드를 `public` 으로 구현할 수 있는가?

예시 답안

```
interface Notifier {  
    void send(String message);  
}
```

```
class EmailNotifier implements Notifier {  
    @Override  
    public void send(String message) {  
        System.out.println("이메일 발송: " + message);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Notifier notifier = new EmailNotifier();  
        notifier.send("회원이입을 축하합니다.");  
    }  
}
```

숙제 운영 방식

쉬운 숙제만 낼 때

1. 클래스와 객체 복습
2. 메서드 복습
3. 생성자 복습
4. 캡슐화 복습
5. getter/setter 복습

중간 난이도까지 낼 때

- 6. 상속 복습
- 7. 오버라이딩 복습
- 8. 다형성 복습

잘하는 학생 추가 문제

- 9. 추상 클래스 복습
- 10. 인터페이스 복습

수업 초반에는 1~5번만 숙제로 내고, 6번 이후는 수업 시간에 같이 풀거나 힌트를 제공하는 방식.